

HDOC Day-14

Write a menu-driven C program using switch-case to input a positive integer and perform one of the following operations:

1. Check whether all digits are unique.
 2. Find the frequency of each digit from 0 to 9.
-

1. Algorithm — Check whether all digits are unique

Step 1: Start

Step 2: Input a positive integer n.

Step 3: Set all elements of freq[10] to 0.

Step 4: Extract the last digit using $\text{digit} = n \% 10$.

Step 5: Increment freq[digit].

Step 6: Remove the last digit using $n = n / 10$.

Step 7: Repeat Steps 4–6 until n becomes 0.

Step 8: Check whether any $\text{freq}[i] > 1$.

Step 9: If yes, print "**Not Unique**", otherwise print "**Unique**".

Step 10: Stop

Test Case Table:

Test Case	Input	Expected Output
1	12345	Unique
2	1234	Unique
3	11234	Not Unique
4	12245	Not Unique
5	987654	Unique

2. Algorithm — Frequency of each digit

Step 1: Start

Step 2: Input a positive integer n.

Step 3: Initialize freq[0] to freq[9] as 0.

Step 4: Extract the last digit using $\text{digit} = n \% 10$.

Step 5: Increment $\text{freq}[\text{digit}]$.

Step 6: Remove the last digit using $n = n / 10$.

Step 7: Repeat Steps 4–6 until n becomes 0.

Step 8: Display the frequency of digits 0 to 9.

Step 9: Stop

Test Case Table:

Test Case	Input	Expected Output
1	112333	Freq_0=0, Freq_1=2, freq_3=3, freq_4=0, freq_5=0, freq_6=0, freq_7=0, freq_8=0, freq_9=0
2	12345	Freq_0=0, freq_1=1, freq_2=1, freq_3=1, freq_4=1, freq_5=1, freq_6=0, freq_7=0, freq_8=0, freq_9=0
3	55555	Freq_5=5 and all other frequencies are zero
4	10020	Freq_0=2, freq_1=1, freq_2=1 and all other frequencies are zero

3. Menu-Driven C Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int choice, n, temp, digit;
```

```
    int freq[10];
```

```
    int i, unique;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    printf("\nMENU\n");
```

```
    printf("1. Check whether all digits are unique\n");
```

```
    printf("2. Find frequency of each digit from 0 to 9\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch(choice)
```

```
{
```

```
        case 1:
```

```
            for(i = 0; i < 10; i++)
```

```
                freq[i] = 0;
```

```
temp = n;
```

```
while(temp > 0)
{
    digit = temp % 10;
    freq[digit]++;
    temp = temp / 10;
}
```

```
unique = 1;
```

```
for(i = 0; i < 10; i++)
{
    if(freq[i] > 1)
    {
        unique = 0;
        break;
    }
}
```

```
if(unique == 1)
    printf("Unique\n");
else
    printf("Not Unique\n");
```

```
break;
```

case 2:

```
for(i = 0; i < 10; i++)
    freq[i] = 0;
```

```
temp = n;
```

```
while(temp > 0)
{
    digit = temp % 10;
    freq[digit]++;
    temp = temp / 10;
}
```

```

        for(i = 0; i < 10; i++)
        {
            printf("freq_%d = %d\n", i, freq[i]);
        }

        break;

    default:
        printf("Invalid choice\n");
    }

    return 0;
}

```

Compilation and Execution:

For Choice 1:

Test Case 1:

```

(kali㉿kali)-[~]
└─$ ./a.out
Enter a positive integer:12345

MENU
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice:1
Unique

```

Test Case 2:

```

(kali㉿kali)-[~]
└─$ ./a.out
Enter a positive integer:1234

MENU
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice:1
Unique

```

Test Case 3:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer:11234  
  
MENU  
1. Check whether all digits are unique  
2. Find frequency of each digit from 0 to 9  
Enter your choice:1  
Not unique
```

Test Case 4:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer:12245  
  
MENU  
1. Check whether all digits are unique  
2. Find frequency of each digit from 0 to 9  
Enter your choice:1  
Not unique
```

Test Case 5:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer:987654  
  
MENU  
1. Check whether all digits are unique  
2. Find frequency of each digit from 0 to 9  
Enter your choice:1  
Unique
```

For Choice 2:

Test Case 1:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer:112333  
  
MENU  
1. Check whether all digits are unique  
2. Find frequency of each digit from 0 to 9  
Enter your choice:2  
freq_0 = 0  
freq_1 = 2  
freq_2 = 1  
freq_3 = 3  
freq_4 = 0  
freq_5 = 0  
freq_6 = 0  
freq_7 = 0  
freq_8 = 0  
freq_9 = 0
```

Test Case 2:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer:12345  
  
MENU  
1. Check whether all digits are unique  
2. Find frequency of each digit from 0 to 9  
Enter your choice:2  
freq_0 = 0  
freq_1 = 1  
freq_2 = 1  
freq_3 = 1  
freq_4 = 1  
freq_5 = 1  
freq_6 = 0  
freq_7 = 0  
freq_8 = 0  
freq_9 = 0
```

Test Case 3:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer:55555  
  
MENU  
1. Check whether all digits are unique  
2. Find frequency of each digit from 0 to 9  
Enter your choice:2  
freq_0 = 0  
freq_1 = 0  
freq_2 = 0  
freq_3 = 0  
freq_4 = 0  
freq_5 = 5  
freq_6 = 0  
freq_7 = 0  
freq_8 = 0  
freq_9 = 0
```

Test Case 4:

```
(kali㉿kali)-[~]  
$ ./a.out
```

Enter a positive integer:10020

MENU

1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9

Enter your choice:2

freq_0 = 3

freq_1 = 1

freq_2 = 1

freq_3 = 0

freq_4 = 0

freq_5 = 0

freq_6 = 0

freq_7 = 0

freq_8 = 0

freq_9 = 0